

Кросплатформне програмування

09

Функціональні інтерфейси,
лямбда-вирази, посилання
на методи та анонімні класи



Функціональні інтерфейси, лямбда-вирази, посилання на методи та анонімні класи

Про що йтиметься:

- Анонімні класи
- Синтаксис лямбда-виразу
- Функціональні інтерфейси
- Використання лямбда-виразів
- Посилання на метод об'єкту
- Посилання на нестатичний метод класу
- Посилання на статичний метод
- Посилання на конструктор



Анонімні класи

Досить часто виникає необхідність у створенні класу, який використовується в програмі один раз. У такому разі нерідко використовують анонімні класи - класи, які не мають "власного імені"

Анонімний клас можна створити шляхом спадкування класу (зазвичай абстрактного) або реалізації інтерфейсу

Анонімний клас на основі абстрактного суперкласу

```
Суперклас змінна = new Суперклас(аргументи) {  
    // Описання методів  
};
```

Анонімний клас на основі інтерфейсу

```
Інтерфейс змінна = new Інтерфейс() {  
    // Описання методів  
};
```



Анонімний клас на основі абстрактного суперкласу

```
// Абстрактний суперклас:
abstract class Base{
    // Текстове поле:
    private String name;
    // Конструктор:
    Base(String txt){
        name=txt;
    }
    // Метод для відображення значення текстового поля:
    void show(){
        System.out.println ("Ім'я об'єкта: "+ name);
    }
    // Оголошення абстрактного методу:
    abstract void hello();
} // Головний клас:
class Demo{
    public static void main(String[] args){
        // Створення об'єкта анонімного класу:
        Base obj=new Base("Червоний"){
            // Описання абстрактного методу з суперкласу:
            void hello(){
                System.out.println ("Об'єкт анонімного класу");
            }
        }; // Завершення інструкції створення об'єкта анонімного класу
```

```
        // Виклик методів з об'єкта, створеного на основі
        // анонімного класу:
        obj.show();
        obj.hello();
        // Створення анонімного об'єкта анонімного класу
        // і виклик з цього об'єкта методу showAll(),
        // описаного в анонімному класі:
        new Base("Зелений"){
            // Описання абстрактного методу з суперкласу:
            void hello(){
                System.out.println ("Анонімний об'єкт");
            }
            // Описання нового методу:
            void showAll(){
                hello();
                show();
            }
        }.showAll(); // Виклик методу
    }
}
```

Результат:

Ім'я об'єкта: Червоний
Об'єкт анонімного класу
Анонімний об'єкт
Ім'я об'єкта: Зелений



Анонімний клас на основі інтерфейсу

```
// Інтерфейс:
interface Base{
    // Метод із кодом за замовчуванням:
    default void show(){
        System.out.println("Інтерфейс Base");
    }
    // Оголошення методу:
    void hello();
}
// Головний клас:
class Demo{
    public static void main(String[] args){
        // Створення об'єкта анонімного класу:
        Base obj=new Base(){
            // Описання методу з інтерфейсу:
            public void hello(){
                System.out.println ("Об'єкт анонімного класу");
            }
        }; // Завершення інструкції створення об'єкта
        // анонімного класу
        // Виклик методів з об'єкта, створеного на основі
        // анонімного класу:
        obj.show();
        obj.hello();
    }
}
```

```
// Створення анонімного об'єкта анонімного класу
// і виклик з цього об'єкта методу showAll(),
// описаного в анонімному класі:
new Base(){
    // Описання методу з інтерфейсу:
    public void hello(){
        System.out.println ("Анонімний об'єкт");
    }
    // Описання нового методу:
    void showAll(){
        hello();
        show();
    }
}.showAll(); // Виклик методу
}
```

Результат:

Інтерфейс Base
Об'єкт анонімного класу
Анонімний об'єкт
Інтерфейс Base



Синтаксис лямбда-виразу

- Лямбда-вираз можна розглядати як блок коду, що визначає дію
- Також можна розглядати лямбда-вираз як визначення анонімного методу - тобто методу, який не має імені (аналогія не зовсім точна, але в багатьох випадках допомагає зрозуміти логіку того, що відбувається)
- Використовуються лямбда-вирази переважно як альтернатива до створення анонімних класів

Шаблон описання лямбда-виразу

```
(аргументи) -> {команди}
```

Приклад описання лямбда-виразу

```
(int x,int y)->{int z= x+y; System.out.println(z);}
```

Приклад описання лямбда-виразу

```
(int x,int y)->{  
    int z=x+y;  
    System.out.println(z);  
}
```



Спрощення синтаксису лямбда-виразів

Є правила , які дозволяють спрощувати синтаксис описання лямбда-виразів:

- Тип аргументу можна не вказувати, якщо він ідентифікується виходячи з контексту команди використання лямбда-виразу
- Якщо в лямбда-виразі один аргумент, тип якого не вказано, круглі дужки можна не використовувати
- Якщо в лямбда-виразі немає аргументів, то використовуються пусті круглі дужки
- Якщо в тілі лямбда-виразу всього одна команда, фігурні дужки можна не використовувати. У випадку, коли єдина команда в тілі лямбда-оператора складається з `return`-інструкції, ключове слово `return` можна не використовувати



Функціональний інтерфейс та лямбда-вирази

Функціональний інтерфейс - це інтерфейс з одним і лише одним абстрактним методом

Лямбда-вираз може бути присвоєно значенням змінній інтерфейсного типу – але за умови, що інтерфейс функціональний, а сигнатура абстрактного методу в функціональному інтерфейсі відповідає параметрам лямбда-виразу (кількість та тип параметрів)



Створюється об'єкт на основі класу (анонімного), що реалізує відповідний інтерфейс, а як код для визначення абстрактного методу використовується код лямбда-виразу



Використання лямбда-виразу

```
// Функціональний інтерфейс:
interface MyNums{
    // Метод із кодом за замовчуванням:
    default void show(int n){
        System.out.println("Аргумент: "+n);
        System.out.println("Результат: "+get(n));
    }
    // Абстрактний метод:
    int get(int n);
}
// Головний клас:
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Присвоєння лямбда-виразу значенням
        // змінній інтерфейсного типу:
        MyNums A=(int n)->{
            // Локальні змінні:
            int k,s=0;
            // Обчислення суми натуральних чисел:
            for(k=1;k<=n;k++){
                s+=k;
            }
            // Результат:
            return s;
        };
    }
}
```

```
System.out.println("Використовуємо лямбда-вираз:");
// Виклик методів з інтерфейсної змінної:
A.show(10);
System.out.println("Перевірка: "+A.get(10));
// Альтернативне посилання об'єкт:
MyNums B=A;
System.out.println("Новий лямбда-вираз:");
// Інтерфейсній змінній значенням присвоюється
// новий лямбда-вираз:
A=n->n*n;
System.out.println("Виклик методу show() з A:");
A.show(10);
System.out.println("Виклик методу show() з B:");
B.show(10);
}
```

**Альтернативний спосіб
створення об'єктів**

Результат:

Використовуємо лямбда-вираз:
Аргумент: 10
Результат: 55
Перевірка: 55
Новий лямбда-вираз:
Виклик методу show() з A:
Аргумент: 10
Результат: 100
Виклик методу show() з B:
Аргумент: 10
Результат: 55





Використання анонімних класів

```
interface MyNums{                                // Інтерфейс
    default void show( int n){
        System.out.println("Аргумент: "+n);
        System.out.println ("Результат: "+get(n));
    }
    int get(int n);
}
class Demo{
    public static void main(String[] args){
        // Присвоєння інтерфейсній змінній посилання
        // на об'єкт анонімного класу:
        MyNums A=new MyNums(){
            public int get(int n){                // Описання методу з інтерфейсу
                int k,s=0;
                for(k=1;k<=n;k++){
                    s+=k;
                }
                return s;
            }
        };
        System.out.println("Використовуємо анонімний клас:");
        A.show(10);
        System.out.println("Перевірка: "+A.get(10));
        MyNums B=A;
        System.out.println("Новий анонімний клас:");
```

```
        // Змінній значенням присвоюється посилання
        // на об'єкт нового анонімного класу:
        A=new MyNums(){
            // Описання методу з інтерфейсу:
            public int get(int n){
                return n*n;
            }
        };
        System.out.println("Виклик методу show() з A:");
        A.show(10);
        System.out.println("Виклик методу show() з B:");
        B.show(10);
    }
}
```

Результат:

Використовуємо анонімний клас:

Аргумент: 10

Результат: 55

Перевірка: 55

Новий анонімний клас:

Виклик методу show() з A:

Аргумент: 10

Результат: 100

Виклик методу show() з B:

Аргумент: 10

Результат: 55

Посилання на метод об'єкта

Посилання на нестатичний метод конкретного об'єкта виконується так: вказується ім'я об'єкта і через оператор :: вказується назва методу

Посилання на метод об'єкта

```
об'єкт::ім'я_методу
```

Якщо посилання на об'єкт виконано в форматі **об'єкт::метод**, і при цьому метод описаний з деякими **аргументами**, то інструкції **об'єкт::метод** відповідає лямбда-вираз виду **аргументи->об'єкт.метод(аргументи)**

```

class MyClass{// Клас
    private int number; // Закрите ціле поле
    MyClass(int n){// Конструктор
        number=n;}
    void set(int n){// Метод для присвоювання значення полю
        number=n;}
    int get(){// Метод для зчитування значення поля
        return number;}
}
interface MyGetter{ // Перший функціональний інтерфейс
    int myget();}
interface MySetter{ // Другий функціональний інтерфейс
    void myset(int n);}
class Demo{ // Головний клас
    public static void main(String[] args){ // Головний метод
        MyClass obj=new MyClass(100); // Створення об'єкта
        System.out.println("Створено об'єкт з полем 100");
        MyGetter A=obj::get; // Використовуємо посилання на методи
        MySetter B=obj::set;
        // Перевіряємо "значення поля" викликом методу myget()
        // з інтерфейсної змінної A:
        System.out.println("Змінна A: "+A.myget());
        // Перевіряємо значення поля об'єкта:
        System.out.println("Змінна obj: "+obj.get());
        // Полю об'єкта присвоюється значення:
        obj.set(200);
        System.out.println("Полю присвоєно значення 200");
        // Перевіряємо "значення поля" викликом методу myget()
        // з інтерфейсної змінної A:
        System.out.println("Змінна A: "+A.myget());
        // Привласнюємо "значення полю" викликом методу myset()
        // з інтерфейсної змінної B:
        B.myset(300);
        System.out.println("Виконано команду B.myset(300)");
        // Перевіряємо "значення поля" викликом методу myget()
        // з інтерфейсної змінної A:
        System.out.println("Змінна A: "+A.myget());
    }
}

```

Посилання на метод об'єкта

Результат:

```

Створено об'єкт з полем 100
Змінна A: 100
Змінна obj: 100
Полю присвоєно значення 200
Змінна A: 200
Виконано команду B.myset(300)
Змінна A: 300
Змінна obj: 300
Створено об'єкт з полем 400
Змінна A: 300
Змінна obj: 400
Виконано команду B.myset(500)
Змінна A: 500
Змінна obj: 400

```

```

// Перевіряємо значення поля об'єкта:
System.out.println("Змінна obj: "+obj.get());
// Створюється новий об'єкт:
obj=new MyClass(400);
System.out.println("Створено об'єкт з полем 400");
// Перевіряємо "значення поля" викликом методу myget()
// з інтерфейсної змінної A:
System.out.println("Змінна A: "+A.myget());
// Перевіряємо значення поля об'єкта:
System.out.println("Змінна obj: "+obj.get());
// Присвоюємо "значення полю" викликом методу myset()
// з інтерфейсної змінної B:
B.myset(500);
System.out.println("Виконано команду B.myset(500)");
// Перевіряємо "значення поля" викликом методу myget()
// з інтерфейсної змінної A:
System.out.println("Змінна A: "+ A.myget());
// Перевіряємо значення поля об'єкта:
System.out.println("Змінна obj: "+obj.get());
}

```



Посилання на нестатичний метод класу

Посилання на нестатичний метод класу виконується в такому вигляді:
вказується ім'я класу, і через оператор `::` вказується назва методу

Посилання на нестатичний метод класу

```
клас :: ім'я_методу
```

Якщо використовується посилання виду `клас::метод`, і при цьому метод нестатичний і описаний з деякими **аргументами**, то вказане посилання еквівалентна лямбда-виразу наступного виду:
`(об'єкт, аргументи) -> об'єкт.метод(аргументи)`



Посилання на нестатичний метод класу

```
class MyClass{ // Клас з полем та методами
    private int number;
    MyClass(int n){
        number=n;
    }
    void set(int n){
        number=n;
    }
    int get(){
        return number;
    }
} // Перший функціональний інтерфейс:
interface MyGetter{
    int myget(MyClass obj);
} // Другий функціональний інтерфейс:
interface MySetter{
    void myset(MyClass obj,int n);
} // Головний клас:
class Demo{
    public static void main(String[] args){
        // Створюється об'єкт:
        MyClass obj=new MyClass(100);
        System.out.println("Створено об'єкт з полем 100");
        // Використовуємо посилання на методи:
        MyGetter A=MyClass::get;
        MySetter B=MyClass::set ;
    }
}
```

```
System.out.println("Змінна A: "+A.myget(obj));
System.out.println("Змінна obj: "+obj.get());
obj.set(200);
System.out.println("Полю присвоєно значення 200");
System.out.println("Змінна A: "+A.myget(obj));
B.myset(obj,300);
System.out.println("Виконано команду B.myset(obj,300)");
System.out.println("Змінна A: "+A.myget(obj));
System.out.println("Змінна obj: "+obj.get());
// Створюється новий об'єкт:
obj=new MyClass(400);
System.out.println("Створено об'єкт з полем 400");
System.out.println("Змінна A: "+ A.myget(obj));
System.out.println("Змінна obj: "+obj.get());
B.myset(obj,500);
System.out.println("Виконано команду B.myset(obj,500)");
System.out.println("Змінна A: "+A.myget(obj));
System.out.println("Змінна obj: "+obj.get());
}
}
```

Результат:

```
Створено об'єкт з полем 100
Змінна A: 100
Змінна obj: 100
Полю присвоєно значення 200
Змінна A: 200
Виконано команду B.myset(obj,300)
Змінна A: 300
Змінна obj: 300
Створено об'єкт з полем 400
Змінна A: 400
Змінна obj: 400
Виконано команду B.myset(obj,500)
Змінна A: 500
Змінна obj: 500
```



Посилання на статичний метод

Посилання на статичний метод класу виконується в такому вигляді: вказується ім'я класу, і через оператор `::` вказується назва методу (тобто виконується так само, як на нестатичний метод класу)

Посилання на статичний метод класу

```
клас::ім'я_методу
```

Якщо використовується посилання на метод виду `клас::метод`, і при цьому метод є статичним і описаний з деякими `аргументами`, то вказане вище посилання еквівалентне лямбда-виразу виду `(аргументи) -> клас.метод(аргументи)`

Наприклад, посилання `System.out::println` еквівалентне лямбда-виразу `t->System.out.println(t)`



Посилання на статичний метод

```
// Клас зі статичними методами:
class MyClass{
    // Методом відображається повідомлення:
    static void show(){
        System.out.println("Метод класу MyClass");
    }
    // Методом обчислюється сума чисел:
    static int sum(int n){
        int k,s=0;
        for(k=1;k<=n;k++){
            s+=k;
        }
        return s;
    }
} // Перший інтерфейс:
interface MyShow{
    void myshow();
} // Другий інтерфейс:
interface MySum{
    int mysum(int n);
} // Третій інтерфейс:
interface MyPrinter{
    void myprint(Object t);
}
```

```
// Головний клас:
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Використання посилань на статичні методи:
        MyShow A=MyClass::show;
        MySum B=MyClass::sum;
        MyPrinter P=System.out::println;
        // Виклик методів з інтерфейсних змінних:
        A.myshow();
        P.myprint("Сума чисел: "+B.mysum(10));
    }
}
```

Результат:

Метод класу MyClass
Сума чисел: 55

Посилання конструктор

Посилання можна виконувати як на метод, так і на конструктор. Синтаксис виконання посилання на конструктор такий: після імені класу вказується оператор `::`, після якого слідує ключове слово `new`

Посилання на конструктор

```
клас :: new
```

Якщо використано посилання на конструктор виду `клас :: new`, то еквівалентом є лямбда-вираз виду `(аргументи) -> new клас (аргументи)`, де мається на увазі, що при створенні об'єкта класу конструктору передаються `аргументи`

Фактично йдеться не просто про конструктор, а про команду створення нового об'єкта за допомогою конструктора. Результатом виразу виду `new клас (аргументи)`, очевидно, є посилання на створений об'єкт - тобто значення типу клас. Такий результат має бути у абстрактного методу з функціонального інтерфейсу, змінній якого значенням надається посилання на конструктор



Посилання на конструктор

```
class MyClass{           // Клас
    private int number;   // Закрите поле
    MyClass(int n){       // Конструктор
        number=n;
    } // Відкриті методи:
    void show(){
        System.out.println("Значення поля: "+number);
    }
    void set(int n){
        number=n;
    }
} // Інтерфейс:
interface MyInterface{
    MyClass create(int n);
} // Головний клас:
class Demo{
    public static void main(String[] args){ // Головний метод
        MyInterface ref=MyClass::new ; // Використання посилання на конструктор
        // Створення об'єкта викликом методу з інтерфейсної змінної:
        MyClass obj=ref.create(100);
        // Виклик методів об'єкта:
        obj.show();
        obj.set(200);
        obj.show();
    }
}
```

Результат:

Значення поля: 100

Значення поля: 200

Посилання на перевантажений метод

- Посилання може виконуватися на перевантажений метод (у класі описано кілька версій методу)
- За посиланням на метод не можна однозначно визначити, про яку версію методу йдеться
- Висновок про те, яку версію методу слід використовувати, робиться на основі інтерфейсної змінної, якій значенням присвоюється посилання на метод

```
// Клас:
class MyClass{
    // Поле:
    int number;
    // Перевантажений метод:
    void set(){
        number=0;
    }
    void set(int n){
        number=n;
    }
}
// Перший інтерфейс:
interface Alpha{
    void none();
}
```

Результат:

Значення поля: 100

Значення поля: 0

```
// Другий інтерфейс:
interface Bravo{
    void one(int n);
}
// Головний клас:
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Створення об'єкта:
        MyClass obj=new MyClass();
        // Використання посилання на перевантажений метод:
        Alpha A=obj::set;
        Bravo B=obj::set;
        // Виклик методу через інтерфейсну змінну:
        B.one(100);
        // Перевірка значення поля об'єкта:
        System.out.println("Значення поля: "+obj.number);
        // Виклик методу через інтерфейсну змінну:
        A.none();
        // Перевірка значення поля об'єкта:
        System.out.println("Значення поля: "+obj.number);
    }
}
```



Лямбда-вираз і поле об'єкта

```
// Функціональний інтерфейс:
interface MyInterface{
    // Метод із цілочисловим аргументом повертає
    // цілочисловий результат:
    int getNumber(int n);
}
// Клас із полем інтерфейсного типу:
class MyClass{
    // Закрите поле інтерфейсного типу:
    private MyInterface ref;
    // Конструктор:
    MyClass(MyInterface mi){
        set(mi);
    }
    // Метод для присвоєння значення полю:
    void set(MyInterface mi){
        ref=mi;
    }
    // Метод із цілочисловим аргументом повертає
    // результатом ціле число:
    int get(int n){
        // Виклик методу з об'єкта, на який посилається
        // поле інтерфейсного типу:
        return ref.getNumber(n);
    }
}
```

```
// Головний клас:
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Створення об'єкта класу з передачею аргументом
        // конструктору лямбда-виразу:
        MyClass obj=new MyClass((int n)->{return n*n;});
        // Перевірка результату:
        System.out.println("Аргумент:");
        for(int k=0;k<=5;k++){
            System.out.print(k+"\t");
        }
        System.out.println("\nАргумент у квадраті:");
        for(int k=0;k<=5;k++){
            System.out.print(obj.get(k)+"\t");
        }
        // Полю об'єкта присвоюється нове значення:
        obj.set((int n)->{return n*n*n;});
        // Перевірка результату:
        System.out.println("\nАргумент у кубі:");
        for(int k=0;k<=5;k++){
            System.out.print(obj.get(k)+"\t");
        }
        System.out.println();
    }
}
```

Результат:

Аргумент:

0 1 2 3 4 5

Аргумент у квадраті:

0 1 4 9 16 25

Аргумент у кубі:

0 1 8 27 64 125

Дякую за увагу!

Запитання?